

ProductionManagementAI — Identity スキーマ — データベース設計書（テーブル定義書）

000_DB — 認証・RBAC 基盤（work-items/WI-001/brief.md INFRA-002）を支える。0002_ADR（docs/en/architecture/0002/0002_ADR_auth-rbac-foundation.md）を実装する。

物理命名規約：snake_case。ai/skills/database-design/SKILL.md（「規約が確定するまでは snake_case の物理名を一貫して使う」）に従う。ASP.NET Core Identity 既定の PascalCase テーブル名（AspNetUsers など）は、AppDbContext の EF Core Fluent API ToTable(...) 呼び出しで snake_case に付け替え、列名は EFCore.NamingConventions（work-items/WI-001/plan.md の認証基盤設計で提案）で一貫して snake_case にする。キーは gen_random_uuid() を既定値とする uuid（クレーム系テーブルは Identity の integer ID キー）、タイムスタンプは timestampz、制約名・インデックス名は EF Core の pk_ / fk_ / ix_ 形式とする。001_DB～003_DB もこの規約に従う。

文書管理（改版履歴）

項目	値
文書 ID	000_DB
システム名	ProductionManagementAI
作業項目	WI-001
データベース	production_management_ai、スキーマ public
DBMS	PostgreSQL 17
文字コード	UTF-8
作成者	Claude（ThanhTN の依頼による）
作成日	2026-09-16
最終更新者	Claude（ThanhTN の依頼による）
最終更新日	2026-09-28

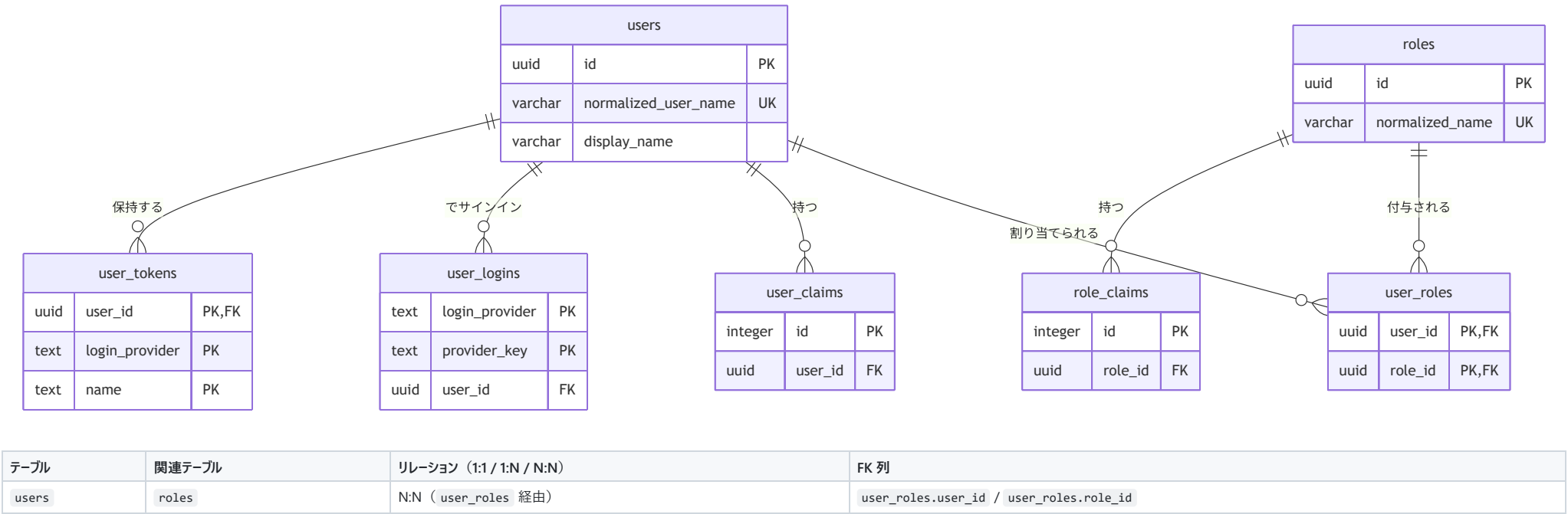
No	対象	作業項目	変更内容	変更日	変更者
1	全体	WI-001	新規作成（DB-001 として）	2026-09-16	Claude （ThanhTN の依頼による）
2	全体	—	テンプレート参照を更新	2026-09-17	ThanhTN
3	未決事項	WI-002	CSRF の決定を決定済みとして記録（WI-002 DEC-020）	2026-09-18	Claude （ThanhTN の依頼による）
4	全体	WI-005	000_DB に改名（RFC 0010）。英語版・日本語版 PDF を追加（RFC 0008）	2026-09-23	Claude （ThanhTN の依頼による）
5	全体	—	テーブル定義テンプレートに合わせて再構成（文書管理、ライフサイクル付きテーブル一覧、テーブルごとの列定義、インデックス一覧・トリガー一覧、データベースロールの節）。InitialIdentitySchema マイグレーションと照合して訂正：users の列順、phone_number の型（varchar(32) ではなく text）、列の既定値（boolean 列と access_failed_count には DB 既定値がなく、アプリケーションが設定する）、FK 名、4 つの FK インデックス、一意性の制約（独立した UNIQUE 制約ではなく一意インデックス）。Identity 既定の 4 テーブルを列ごとに記載。user_claims と role_claims をサインイン時に読むものとして記録	2026-09-28	Claude （ThanhTN の依頼による）

テーブル一覧

No	スキーマ	物理名	論理名	概要	PK 以外のインデックス	トリガー	作成日	スキーマ変更日	削除日	利用状況
1	public	users	ユーザー	アプリケーションのユーザーアカウント（Identity IdentityUser<Guid> + アプリ固有項目）	有	無	2026-09-16	—	—	使用中
2	public	roles	ロール	RBAC ロール（Identity IdentityRole<Guid>）。Admin、Operator を投入	有	無	2026-09-16	—	—	使用中
3	public	user_roles	ユーザーロール	ユーザーとロールの多対多の中間テーブル	有	無	2026-09-16	—	—	使用中
4	public	user_claims	ユーザークレーム	Identity 既定 — ユーザーごとのクレーム	有	無	2026-09-16	—	—	使用中 — サインイン時に読む。空で、書き込むものはない
5	public	role_claims	ロールクレーム	Identity 既定 — ロールごとのクレーム	有	無	2026-09-16	—	—	使用中 — サインイン時に読む。空で、書き込むものはない
6	public	user_logins	ユーザーログイン	Identity 既定 — 外部ログインプロバイダー	有	無	2026-09-16	—	—	未使用（外部ログインは対象外）
7	public	user_tokens	ユーザートークン	Identity 既定 — ユーザーごとのトークン（パスワードリセットなど）	無	無	2026-09-16	—	—	未使用（拡張用に保持）

「作成日」は、そのテーブルを作成したマイグレーション（InitialIdentitySchema、2026-09-16）の日付。

ER 図とリレーション



テーブル	関連テーブル	リレーション (1:1 / 1:N / N:N)	FK 列
users	user_claims	1:N (0 件もありうる)	user_claims.user_id
users	user_logins	1:N (0 件もありうる)	user_logins.user_id
users	user_tokens	1:N (0 件もありうる)	user_tokens.user_id
roles	role_claims	1:N (0 件もありうる)	role_claims.role_id

テーブル定義

Identity が管理する列の検証はすべて ASP.NET Core Identity 自身（`userManager` / `roleManager` とそのオプション）が行う。これらのテーブルに書き込む画面はない。DB 既定値のない列は、挿入時にアプリケーションが値を設定する（Identity のエンティティ既定値、または `AppUser` の初期化子）。

ユーザー（users）

項目	値
論理名	ユーザー
物理名	users
スキーマ	public
概要	アプリケーションのユーザーアカウント 1 件につき 1 行。ASP.NET Core Identity（ <code>userManager</code> ）経由でのみ書き込む： <code>IdentitySeeder</code> による開発用シード管理者と、ログイン時の管理情報（ロックアウト、失敗回数、セキュリティスタンプ）。列は <code>IdentityUser<Guid></code> に、アプリ固有の <code>display_name</code> 、 <code>is_active</code> 、 <code>created_at_utc</code> （ <code>AppUser</code> ）を加えたもの
想定件数	少数・上限あり — 工場スタッフのアカウント、数十行
保持・削除	保持。削除機能はない。ユーザーを削除すると、ロール割り当て・クレーム・ログイン・トークンが連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	Id	id	uuid	—	yes	gen_random_uuid()	○			—	○	Identity <code>IdentityUser<Guid></code> のキー	—	<code>GET /api/auth/me</code> で返す
2	DisplayName	display_name	varchar	200	yes	なし				作成時にアプリケーションが設定	○	UI ヘッダーに表示する名前	—	アプリ固有の追加列
3	IsActive	is_active	boolean	—	yes	なし（アプリケーションが <code>true</code> を設定）				—		アカウント有効フラグ	<code>true</code> : 有効、 <code>false</code> : 無効	アプリ固有の追加列。将来のアカウント無効化のために予約。まだどの要件も使っていない
4	CreatedAtUtc	created_at_utc	timestampztz	—	yes	now()				—	○	アカウント作成日時（UTC）	—	アプリ固有の追加列
5	UserName	user_name	varchar	256	no	なし				Identity	○	ログイン名	—	<code>POST /api/auth/login</code> → <code>userName</code>
6	NormalizedUserName	normalized_user_name	varchar	256	no	なし		○		Identity	○	大文字小文字を区別しない検索用の、大文字化したユーザー名	—	<code>ix_users_normalized_user_name</code> で一意
7	Email	email	varchar	256	no	なし				Identity		メールアドレス	—	Identity 既定。メールを使う機能はまだない

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
8	NormalizedEmail	normalized_email	varchar	256	no	なし				Identity		検索用の大文字化したメールアドレス	—	一意ではない (RequireUniqueEmail は無効)
9	EmailConfirmed	email_confirmed	boolean	—	yes	なし (アプリケーションが false を設定)				Identity		メール確認済みフラグ	—	メール確認フローは対象外
10	PasswordHash	password_hash	text	—	no	なし				Identity	○	Identity が管理するパスワードハッシュ	—	Identity API 以外で読み書きしない
11	SecurityStamp	security_stamp	text	—	no	なし				Identity	○	資格情報の変更時に変わり、既存セッションを無効にする	—	Identity 既定
12	ConcurrencyStamp	concurrency_stamp	text	—	no	なし				Identity	○	Identity の更新に使う楽観的同時実行トークン	—	Identity 既定
13	PhoneNumber	phone_number	text	—	no	なし				Identity		電話番号	—	Identity 既定。未使用
14	PhoneNumberConfirmed	phone_number_confirmed	boolean	—	yes	なし (アプリケーションが false を設定)				Identity		電話番号確認済みフラグ	—	Identity 既定。未使用
15	TwoFactorEnabled	two_factor_enabled	boolean	—	yes	なし (アプリケーションが false を設定)				Identity		2 要素認証フラグ	—	2 要素認証フローは対象外
16	LockoutEnd	lockout_end	timestamptz	—	no	なし				Identity	○	現在のロックアウトの終了日時 (ある場合)	—	0002_ADR の STRIDE 対策 (ロックアウト) を支える
17	LockoutEnabled	lockout_enabled	boolean	—	yes	なし (新規ユーザーには Identity が true を設定)				Identity	○	このユーザーをロックアウトできるか	—	Identity 既定
18	AccessFailedCount	access_failed_count	integer	—	yes	なし (アプリケーションが 0 を設定)				Identity	○	連続したサインイン失敗回数	—	サインイン成功でリセット

- **利用パターン (パターン)** : 該当なし — 1 種類のレコードのみ。
- **CSV 入出力 (DL対象 / UL更新)** : 該当なし — CSV 入出力はない。
- **初期データ**: 開発用の管理者ユーザー 1 件 (admin、表示名 Seed Admin)。SEED_ADMIN_PASSWORD が設定されているとき、マイグレーションではなく起動時に IdentitySeeder が作成する (0002_ADR)。

ロール (roles)

項目	値
論理名	ロール
物理名	roles
スキーマ	public
概要	RBAC ロール (Identity IdentityRole<Guid>)。RoleManager 経由で IdentitySeeder だけが書き込む
想定件数	少数・上限あり — 仮のロール 2 行
保持・削除	保持。ロール削除を提供する機能はない

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	Id	id	uuid	—	yes	gen_random_uuid()	○			—	○	Identity IdentityRole<Guid> のキー	—	
2	Name	name	varchar	256	no	なし				Identity	○	ロール名	Admin、Operator（仮のロール、0002_ADR）	GET /api/auth/me → roles で返す
3	NormalizedName	normalized_name	varchar	256	no	なし		○		Identity	○	検索用の大文字化したロール名	—	ix_roles_normalized_name で一意
4	ConcurrencyStamp	concurrency_stamp	text	—	no	なし				Identity	○	楽観的同時実行トークン	—	Identity 既定

- 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- 初期データ：Admin と Operator。存在しなければ、マイグレーションではなく起動時に IdentitySeeder が作成する。

ユーザーロール（user_roles）

項目	値
論理名	ユーザーロール
物理名	user_roles
スキーマ	public
概要	ユーザーに割り当てたロール 1 件につき 1 行（Identity IdentityUserRole<Guid>）
想定件数	少数・上限あり — 最大でユーザー数 × ロール数
保持・削除	ユーザーからロールを外すと物理削除。users と roles から連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	UserId	user_id	uuid	—	yes	なし	○		users.id	—	○	割り当て先ユーザー	—	role_id との複合 PK
2	RoleId	role_id	uuid	—	yes	なし	○		roles.id	—	○	割り当てるロール	—	user_id との複合 PK

- 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- 初期データ：シード管理者への Admin の割り当て。IdentitySeeder が作成する。

ユーザークレーム（user_claims）

項目	値
論理名	ユーザークレーム
物理名	user_claims
スキーマ	public
概要	ユーザーごとのクレームを持つ Identity 既定のテーブル。AddIdentity<AppUser, AppRole> が登録する Identity の UserClaimsPrincipalFactory が、サインインのたびにサインインしたユーザーについてこのテーブルを読む。書き込む機能はないので空のまま。コストはほぼゼロであり、将来クレームが必要になってもスキーマ変更が不要なように保持する（work-items/WI-001/plan.md 認証基盤設計）

項目	値
想定件数	0 件
保持・削除	Identity が物理削除する。users から連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	Id	id	integer	—	yes	ID 列 (GENERATED BY DEFAULT AS IDENTITY)	○			—		代理キー	—	シーケンス user_claims_id_seq
2	UserId	user_id	uuid	—	yes	なし			users.id	—	○	所有ユーザー	—	サインイン時にユーザー単位でクレームを読む
3	ClaimType	claim_type	text	—	no	なし				Identity	○	クレーム種別 URI	—	
4	ClaimValue	claim_value	text	—	no	なし				Identity	○	クレーム値	—	

- ・ 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- ・ CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- ・ 初期データ: なし。

ロールクレーム（role_claims）

項目	値
論理名	ロールクレーム
物理名	role_claims
スキーマ	public
概要	ロールごとのクレームを持つ Identity 既定のテーブル。サインインのたびに、ユーザーの各ロールについて読む（ user_claims と同じプリンシパルファクトリー）。書き込む機能はないので空のまま
想定件数	0 件
保持・削除	Identity が物理削除する。roles から連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	Id	id	integer	—	yes	ID 列 (GENERATED BY DEFAULT AS IDENTITY)	○			—		代理キー	—	シーケンス role_claims_id_seq
2	RoleId	role_id	uuid	—	yes	なし			roles.id	—	○	所有ロール	—	サインイン時にロール単位でクレームを読む
3	ClaimType	claim_type	text	—	no	なし				Identity	○	クレーム種別 URI	—	
4	ClaimValue	claim_value	text	—	no	なし				Identity	○	クレーム値	—	

- ・ 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- ・ CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- ・ 初期データ: なし。

ユーザーログイン（user_logins）

項目	値
論理名	ユーザーログイン

項目	値
物理名	user_logins
スキーマ	public
概要	ユーザーと外部ログインプロバイダーを結び付ける Identity 既定のテーブル。未使用：外部ログインは対象外
想定件数	0 件
保持・削除	Identity が物理削除する。users から連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	LoginProvider	login_provider	text	—	yes	なし	○			Identity		外部プロバイダー名	—	provider_key との複合 PK
2	ProviderKey	provider_key	text	—	yes	なし	○			Identity		プロバイダー側でのユーザーのキー	—	login_provider との複合 PK
3	ProviderDisplayName	provider_display_name	text	—	no	なし				Identity		表示用のプロバイダー名	—	
4	UserId	user_id	uuid	—	yes	なし			users.id	—		所有ユーザー	—	

- ・ 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- ・ CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- ・ 初期データ: なし。

ユーザートークン（user_tokens）

項目	値
論理名	ユーザートークン
物理名	user_tokens
スキーマ	public
概要	ユーザーごとのトークン（パスワードリセット、認証アプリのキーなど）を持つ Identity 既定のテーブル。まだどの機能も使っていない
想定件数	0 件
保持・削除	Identity が物理削除する。users から連鎖削除される

No	項目名	物理名	データ型	桁数	NOT NULL	既定値	PK	UK	FK	検証	使用	概要説明	区分値	備考
1	UserId	user_id	uuid	—	yes	なし	○		users.id	—		所有ユーザー	—	login_provider 、 name との複合 PK
2	LoginProvider	login_provider	text	—	yes	なし	○			Identity		トークンの発行元	—	
3	Name	name	text	—	yes	なし	○			Identity		トークン名	—	
4	Value	value	text	—	no	なし				Identity		トークン値	—	

- ・ 利用パターン（パターン）：該当なし — 1 種類のレコードのみ。
- ・ CSV 入出力（DL対象 / UL更新）：該当なし — CSV 入出力はない。
- ・ 初期データ: なし。

インデックス一覧

すべて InitialIdentitySchema （2026-09-16）が、新規の空テーブルに作成したもの。したがって CONCURRENTLY は不要だった。

No	テーブル	インデックス名	方式	一意	列とソート順	部分インデックス条件 / INCLUDE	CONCURRENTLY で作成	作成日	削除日	根拠（アクセスパターン）
1	users	pk_users	btree	yes	id ASC	—	no — 新規テーブル	2026-09-16	—	主キー。ID によるユーザー読み込み（Cookie のプリンシパル）
2	users	ix_users_normalized_user_name	btree	yes	normalized_user_name ASC	—	no — 新規テーブル	2026-09-16	—	ログインは正規化ユーザー名で検索する。Identity が一意性を要求する
3	users	ix_users_normalized_email	btree	no	normalized_email ASC	—	no — 新規テーブル	2026-09-16	—	Identity のメール検索。RequireUniqueEmail が無効なので一意ではない（ブリーフはメールによるログインを要求していない）
4	roles	pk_roles	btree	yes	id ASC	—	no — 新規テーブル	2026-09-16	—	主キー
5	roles	ix_roles_normalized_name	btree	yes	normalized_name ASC	—	no — 新規テーブル	2026-09-16	—	名前によるロール検索。初期データ投入を幂等に保つ
6	user_roles	pk_user_roles	btree	yes	user_id ASC, role_id ASC	—	no — 新規テーブル	2026-09-16	—	サインイン時と GET /api/auth/me での「このユーザーのロールは何か」
7	user_roles	ix_user_roles_role_id	btree	no	role_id ASC	—	no — 新規テーブル	2026-09-16	—	EF Core の FK 規約。roles からの連鎖削除と「ロールに属するユーザー」に使う
8	user_claims	pk_user_claims	btree	yes	id ASC	—	no — 新規テーブル	2026-09-16	—	主キー
9	user_claims	ix_user_claims_user_id	btree	no	user_id ASC	—	no — 新規テーブル	2026-09-16	—	EF Core の FK 規約。サインイン時にユーザーのクレームを読み込む
10	role_claims	pk_role_claims	btree	yes	id ASC	—	no — 新規テーブル	2026-09-16	—	主キー
11	role_claims	ix_role_claims_role_id	btree	no	role_id ASC	—	no — 新規テーブル	2026-09-16	—	EF Core の FK 規約。サインイン時に各ロールのクレームを読み込む
12	user_logins	pk_user_logins	btree	yes	login_provider ASC, provider_key ASC	—	no — 新規テーブル	2026-09-16	—	外部ログインからユーザーを検索
13	user_logins	ix_user_logins_user_id	btree	no	user_id ASC	—	no — 新規テーブル	2026-09-16	—	EF Core の FK 規約。ユーザー単位のログイン
14	user_tokens	pk_user_tokens	btree	yes	user_id ASC, login_provider ASC, name ASC	—	no — 新規テーブル	2026-09-16	—	ユーザー・プロバイダー・名前によるトークン検索。先頭列は FK にも使える

トリガー一覧

なし — トリガーはない。

制約

制約	種別（PK/FK/UNIQUE/CHECK/EXCLUDE）	テーブル.列	参照先	ルール
pk_users	PK	users.id	—	—
pk_roles	PK	roles.id	—	—
pk_user_roles	PK	user_roles.(user_id, role_id)	—	—
pk_user_claims	PK	user_claims.id	—	—
pk_role_claims	PK	role_claims.id	—	—

制約	種別 (PK/FK/UNIQUE/CHECK/EXCLUDE)	テーブル列	参照先	ルール
pk_user_logins	PK	user_logins.(login_provider, provider_key)	—	—
pk_user_tokens	PK	user_tokens.(user_id, login_provider, name)	—	—
fk_user_roles_users_user_id	FK	user_roles.user_id	users.id	ON DELETE CASCADE — ユーザーを削除するとそのロール割り当ても削除される
fk_user_roles_roles_role_id	FK	user_roles.role_id	roles.id	ON DELETE CASCADE — ロールを削除するとその割り当ても削除される（ロール削除自体を提供する機能はまだない）
fk_user_claims_users_user_id	FK	user_claims.user_id	users.id	ON DELETE CASCADE
fk_role_claims_roles_role_id	FK	role_claims.role_id	roles.id	ON DELETE CASCADE
fk_user_logins_users_user_id	FK	user_logins.user_id	users.id	ON DELETE CASCADE
fk_user_tokens_users_user_id	FK	user_tokens.user_id	users.id	ON DELETE CASCADE

users.normalized_user_name と roles.normalized_name の一意性は、一意インデックス ix_users_normalized_user_name と ix_roles_normalized_name（インデックス一覧 No. 2 と 5）が保証する。独立した UNIQUE 制約はない。どちらの列も NULL 許容で、PostgreSQL は一意インデックスで NULL 同士を別の値として扱うが、Identity は常にユーザー名・ロール名からこれらの列を設定するので、問題は起きない。

データベースロールと権限

WI-001 でこれらのテーブルを作成した時点では、バックエンドはすべてをオーナーのログインで実行していた。WI-002（001_DB、DEC-016）以降、バックエンドは制限付きの pmai_app ログインで動く。AddProductionOrders マイグレーションが、このログインにこれらのテーブルへの次の権限を付与する：

ロール	使用者	これらのテーブルへの権限	理由
pmai_app（実行時ログイン）	実行時の API（IdentitySeeder を含む）	7 テーブルすべてに SELECT, INSERT, UPDATE。user_roles、user_claims、user_logins、user_tokens に DELETE。シーケンス user_claims_id_seq、role_claims_id_seq に USAGE, SELECT	UserManager、RoleManager、SignInManager が発行する操作のため：サインイン時の管理情報で users を更新し、初期データ投入でユーザー・ロール・割り当てを挿入し、Identity は 4 つの子テーブルから行を削除する。users / roles への DELETE、DDL、TRUNCATE は付与しない
オーナー（PGRES_USER）	マイグレーションのみ（dotnet ef database update）	オーナー — DDL	スキーマの作成と変更

API / DD 対応

テーブル列	DD 項目	API 項目
users.id	未作成（WI-001 には DD がない。認証エンドポイントは 0002_ADR で直接規定）	GET /api/auth/me → id
users.user_name	—	GET /api/auth/me → userName、POST /api/auth/login リクエスト → userName
users.display_name	—	GET /api/auth/me → displayName
roles.name（user_roles 経由）	—	GET /api/auth/me → roles: string[]

マイグレーションの影響と復旧限界

マイグレーション	種別	対象テーブル	稼働中のテーブルか	手順
InitialIdentitySchema（2026-09-16）	追加	上記 7 テーブルすべて（新規作成）	no — プロジェクト最初のマイグレーション	単一ステップ

- データ復旧限界: 作成時点では該当なし — 既存の行はなかった。現在これを戻すと、すべてのユーザーアカウントとロール割り当てが削除される。再作成されるのはシード管理者だけである。
- ロールバック計画: `dotnet ef database update 0` でこれらのテーブルを削除する。WI-002 以降、後続のマイグレーションがこれらのテーブルに権限を付与し依存しているため、先に後続のマイグレーションをすべて戻した後でのみ可能。

未決事項

決定事項	選択肢	推奨	状態
状態を変更するエンドポイントに、（Cookie の SameSite 属性に加えて）CSRF トークンが必要か	SameSite=Lax のみ／ SameSite=Lax + 偽造防止トークン	SameSite=Lax のみ — 追加のトークンなし	決定済み — work-items/WI-002/decisions.md DEC-020（2026-09-18）
email_confirmed / two_factor_enabled / phone_number* 列を将来の要件が使うことがあるか	未使用のまま保持／モデルから削除	保持（コストはほぼゼロで、後の破壊的マイグレーションを避けられる） — 決定済み、users テーブルの備考を参照	決定済み — work-items/WI-001/decisions.md（認証基盤設計）を参照